Chapter 11 worked solutions

DS11-E01 • B-tree insertion and deletion

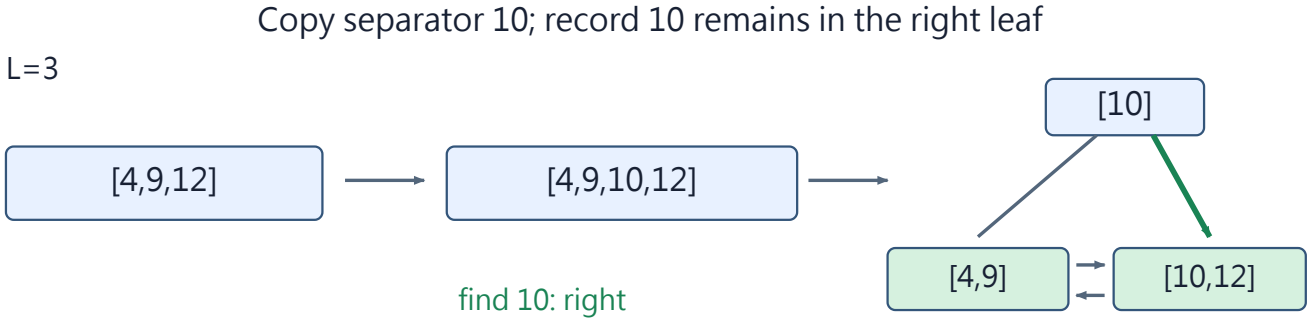
After 10,20,5 the root is `[5,10,20]`. Split before inserting 6, promoting 10 and leaving leaves `[5]`, `[20]`. Insert 6 into `[5,6]`. Insert 12 and 30 into the right leaf, obtaining `[12,20,30]`. Split it before inserting 25 and promote 20. The root is `[10,20]`, with leaves `[5,6]`, `[12]`, `[25,30]`. Before deleting 12, the middle leaf has t−1=1 key while the left sibling has two. Move 6 up to replace parent key 10 and move 10 down to the middle leaf. Delete 12. Final root `[6,20]` has leaves `[5]`, `[10]`, `[25,30]`. Every nonroot leaf contains one through three keys, the root has two keys and three children, and every leaf has depth 1.

DS11-E02 • Height and I/O

Depth d≥1 has at least $2t^{d-1}$ nodes, each with at least t−1 keys. Total keys are at least $1 + 2(t-1)(1 + t + \cdots + t^{h-1}) = 2t^h - 1$. With t=64, height 3 needs at least 524,287 keys, whereas height 4 needs at least 33,554,431. The largest permitted integer height is 3. The minimum-capacity inequality alone does not exclude h=2. Its maximum capacity is $(2t)^3 - 1 = 128^3 - 1 = 2{,}097{,}151$, enough for 1,000,000, so height 2 is also feasible. Height counts edges. A root-to-leaf path contains h+1 pages, less a root read when it is cached.

DS11-E03 • B+ separators and deletion

The temporary leaf `[4,9,10,12]` splits into `[4,9]` and `[10,12]`. Copy separator 10 into the parent while retaining record 10 in the right leaf. Left.next points right. With a doubly linked chain, right.prev points left. Equality routes right to the record. In the separate L=2 state, deleting 10 leaves `[12]`, satisfying minimum ⌈2/2⌉=1. No borrowing or merge is required, and leaf links stay unchanged. The exact right-minimum convention changes root separator 10 to 12. Leaving it unchanged violates that representation even if some searches still happen to work.



DS11-E04 • Range-query assumptions

The premise $M \geq 3$ gives $a = \lceil M/2 \rceil \geq 2$, so the logarithm base is valid. For h≥1, the root has at least two children and later internal pages at least a. There are at least $2a^{h-1}$ leaves, so $n \geq 2\ell a^{h-1}$ and $h \leq 1 + \log_a(n/(2\ell))$. Treat h=0 as one page. Location costs O(h+1); except at the two partial ends, every scanned leaf contributes at least ell outputs. Thus index cost is $O(h + 1 + k/\ell)$. With M,L=Θ(B), fixed-size entries/records, and one page per node, this simplifies to $O(1 + \log_B n + k/B)$. Fanout controls location; minimum leaf occupancy controls scanning.

Choose q≥3,M=3,$L = B = 2^q$, two children per internal page, and 2^q full B-record leaves. Then n=B^2 and h=q. Finding the largest key reads q+1 pages, although $\log_B n = 2$. L=Θ(B) alone does not make internal branching grow with B.

Scattered complete records add D data-page reads, worst-case D=k. Clustering in key order at Θ(B) records per data page permits D=O(1+k/B). Producing k outputs independently requires Ω(k) elementary work.

The chapter stores (child_page_id,cached_min) inside one parent page. Changed children return minima along a (page_id,index) ancestor stack; refresh does not fetch all child pages. Internal splitting moves complete cached entries, without persistent child-parent rewrites. Constant page accesses per level give O(h+1) update I/Os. Separate child-resident minima or parent fields can instead add Θ(M) accesses to one refresh/split. Moving entries within a page can still take O(M) CPU work.

DS11-E05 • Incompatible order conventions

Minimum degree t is an integer, so t=2.5 is invalid. If A also requires at least ⌈m/2⌉=3 children and at most five, its nonroot nodes contain two through four keys. With t=2, B allows one through three keys. With t=3, it allows two through five. Neither matches. A can split after overflowing to five keys, promote the middle key, and retain two on each side. Our top-down procedure splits a full 2t−1-key node into t−1 keys per side before insertion. Insertion timing, maximum capacity, and split allocation must be specified together.

DS11-E06 • B+ underflow, exact separators, and a range

Each nonroot leaf needs ceil(3/2)=2 records. Deleting 10 leaves [12], while both siblings have exactly two records and cannot lend. Merge with [4,9] to obtain [4,9,12], and reconnect [4,9,12]↔[14,16]. The root now has two children and separator [14], the right minimum.

Deleting 14 leaves [16]. The left leaf lends 12, giving [4,9]↔[12,16], each with two records; update root separator 14→12. Range [9,14] descends left at 12, emits 9, follows next, emits 12, then stops at 16>14. Output [9,12] excludes the inspected stopping key.

Deleting 4 leaves [9]. The right sibling cannot lend: merge into [9,12,16], which fits L=3. The root has one child and no separator; replace it with this leaf. Discard routing separator 12, whose record remains in the leaf; adding it would duplicate 12. In a B-tree the parent key carries a record and must move into the merged child; discarding it loses data.

DS11-E07 • Complete internal-repair trace

> Nonroot minima: two records per leaf, two children per internal page. Delete 1: A=[3]; B=[5,7] cannot lend. Merge as AB=[3,5,7], retaining A's ID; free B. Set AB.next=C, C.prev=AB. P's single child requires further repair.

Q lends (C,9), producing the arrays below. Separators: P=[9], Q=[17], R=[13]. Both internal pages have two children; AB has three records, other leaves two. Chain AB↔C↔D↔E stays unchanged. Search 9 follows R→P→C: 9<13, then equality routes right at 9.

Stage	P.entries	Q.entries	R.entries
Initial	(A,1),(B,5)	(C,9),(D,13),(E,17)	(P,1),(Q,9)
After leaf merge	(AB,3)	Unchanged	(P,3),(Q,9)
After child transfer	(AB,3),(C,9)	(D,13),(E,17)	(P,3),(Q,13)

Without E, Q's two children cannot lend. Merge as P=[(AB,3),(C,9),(D,13)], separators [9,13]: three children fit M=4. Set root_id=P; free Q,R. Height becomes one. Chain AB↔C↔D contains [3,5,7,9,11,13,15], exactly the original records minus 1.

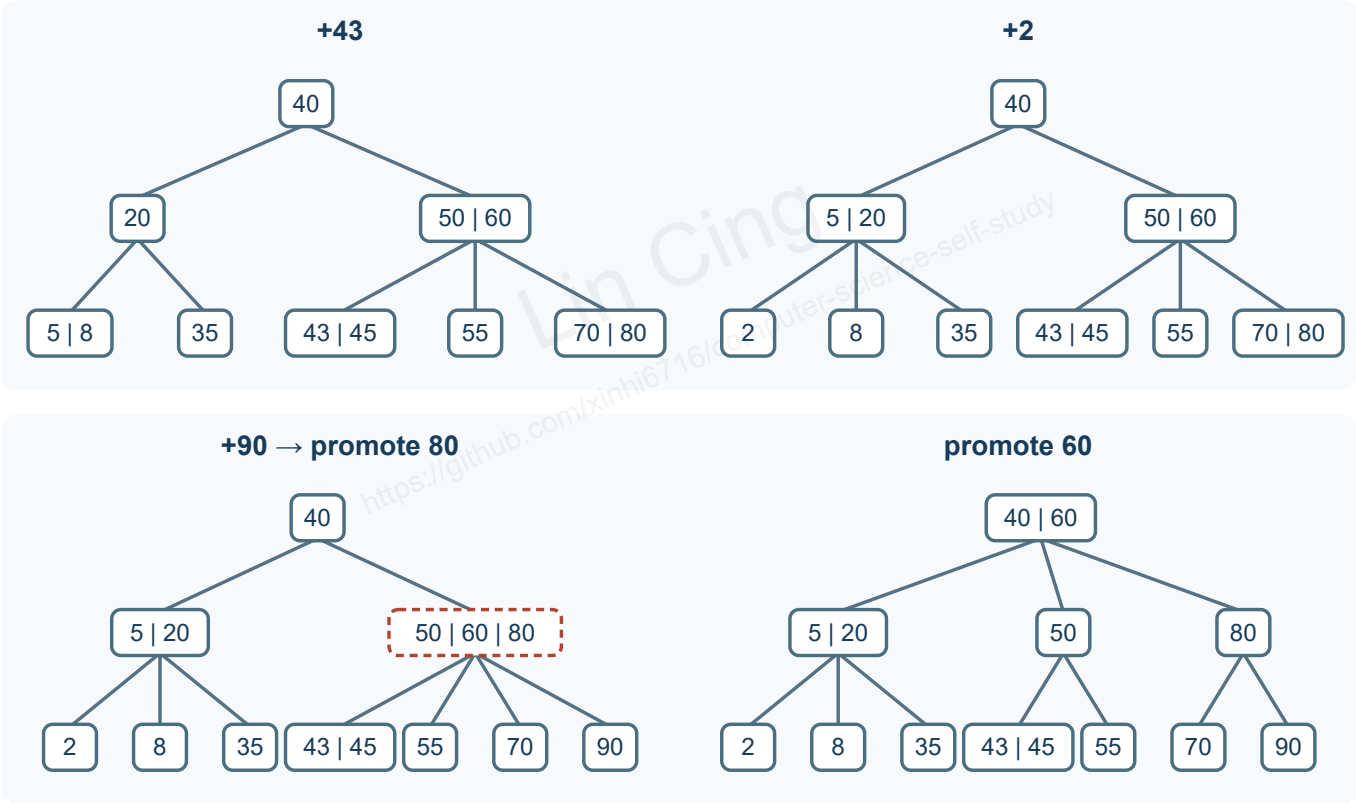
Range [7,14] emits 7 in AB, 9,11 in C, then 13 in D; stop at 15>14. Output: [7,9,11,13]. The root has three children and two separators. Record 9 remains in C: inserting old separator 9 into a leaf duplicates it. B+ internal merging adds no records; a B-tree parent record must instead move into the merged child.

DS11-E08 • B-tree capacities and split timing

Order 3 permits at most three children and at least one non-root key; t=2 permits at most four children and at least one non-root key. A full order-3 node has two keys; promoting one leaves just one key for two non-root children. After insertion the temporary keys are [a,b,c]; promote b and leave [a], [c]. If the parent overflows, continue upward; only a root split increases height.

DS11-E09 • Consecutive 2–3 tree insertions

Inserting 43 changes only one leaf. Inserting 2 promotes 5 into [20], producing [5,20]. Inserting 90 promotes 80, then promotes 60 from the overflowing parent into the root. Dashed nodes denote temporary overflow, not a valid completed node.



The final inorder sequence is 2, 5, 8, 20, 35, 40, 43, 45, 50, 55, 60, 70, 80, 90: 14 distinct keys. Every leaf has depth 2 and each non-root node has one or two keys.

DS11-E10 • Page reads and writes during a split

Search reads the root [60], left internal page [10], and leaf [5,8]: three page reads. Insertion temporarily yields [2,5,8], promotes 5, changes the original leaf to [2], creates leaf [8], and changes the left internal page to [5,10] with children [2], [8], [20]. Writing the original leaf, new leaf and modified parent costs three page writes, six page I/Os in total. The root is unchanged, the right subtree is untouched, and the new page needs no read. Different buffering, allocation or writeback policies can change the count.

DS11-E11 • Height bounds for general order

For $h \geq 1$, minimum node counts by depth are $1, 2, 2q, \ldots, 2q^{h-1}$, giving $1+2(q^h-1)/(q-1)$ nodes. The root has at least one key and every non-root node at least $q-1$, yielding $1+(q-1) \cdot 2(q^h-1)/(q-1)=2q^h-1$ keys. A full tree has $1, m, \ldots, m^h$ nodes by depth, or $(m^{h+1}-1)/(m-1)$ nodes; with $m-1$ keys each it has $m^{h+1}-1$ keys. For $m=3$, $q=2$, $h=2$ the four values are 7, 13, 7, 26. At $h=0$ there is one nonempty root leaf with 1 to $m-1$ keys; the same formulas give those endpoints.

References and attribution

Lin Cing, handwritten data structures notes. Pat Morin, Open Data Structures, § 14.2. PostgreSQL 18, B-tree implementation.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study